

# Lecture 6

## The Fast Fourier Transform (FFT)

---

CSEN 1038 Advanced Data Structures and Algorithms

# Table of contents

1. Introduction
2. Polynomial Representations
3. Fast Polynomial Multiplication

# Introduction

---

- **Intuition:** The Fourier transform gives the frequency spectrum of a signal.
- The discrete Fourier transform has many applications in science and engineering.
- It is used extensively in:
  - Digital Signal Processing (DSP)
  - Spectroscopy

# Polynomial Representations

---

# Polynomial Representations

- A polynomial  $A$  can be written as:

$$A(x) = a_0 + a_1x + a_2x^2 + \cdots + a_{n-1}x^{n-1}$$

- The degree is the highest power with a non-zero coefficient.
- We will focus on two representations:
  - Coefficient representation
  - Point (sample) representation

# Coefficient vs. Sample Representation

- **Coefficient representation:** Store the vector of coefficients  $(a_0, a_1, \dots, a_{n-1})$ .
- **Point (sample) representation:** Store the value of the polynomial at  $n$  distinct points  $x_0, x_1, \dots, x_{n-1}$ .

$$(x_0, y_0), (x_1, y_1), \dots, (x_{n-1}, y_{n-1})$$

# Polynomial Addition

- Given two polynomials  $A$  and  $B$ , we define their sum as a polynomial  $C$  such that  $C(x) = A(x) + B(x)$  for all  $x$ .
- **Addition in Coefficient Representation:**
  - $c_j = a_j + b_j$
  - Time Complexity:  $\mathcal{O}(n)$
- **Addition in Sample Representation:**
  - $y_{c,j} = y_{a,j} + y_{b,j}$  (*assuming we sample at the same points*)
  - Time Complexity:  $\mathcal{O}(n)$



- **Coefficient Representation:**

- $A(x) = a_0 + x(a_1 + x(a_2 + \dots x(a_{n-1}) \dots))$
- Time Complexity:  $\mathcal{O}(n)$  (using Horner's method)

- **Sample Representation:**

- There is no way to get the value of  $A(x)$  at an arbitrary point in the sample representation.
- We must first convert to coefficient form using interpolation, then evaluate. The fastest general interpolation algorithms are  $\mathcal{O}(n^2)$ .

# Polynomial Multiplication

- Given two polynomials  $A$  and  $B$ , we define their product as  $C$ , such that  $C(x) = A(x) \cdot B(x)$  for all  $x$ .
- Example:**

$$A(x) = 1 + 2x$$

$$B(x) = 2 + x + 3x^2$$

$$C(x) = 2 + 3x + 5x^2 + 6x^3$$

- Coefficient Representation:**
  - $c_m = \sum_{j+k=m} a_j \cdot b_k$  (The convolution of the two coefficient vectors!)
  - Time Complexity:  $\mathcal{O}(n^2)$
- Sample Representation:**
  - $y_{c,j} = y_{a,j} \cdot y_{b,j}$  (assuming we sample at the same points)
  - Time Complexity:  $\mathcal{O}(n)$

# Operations Summary

| Operation      | Coefficient Rep.   | Sample Rep.        |
|----------------|--------------------|--------------------|
| Addition       | $\mathcal{O}(n)$   | $\mathcal{O}(n)$   |
| Multiplication | $\mathcal{O}(n^2)$ | $\mathcal{O}(n)$   |
| Evaluation     | $\mathcal{O}(n)$   | $\mathcal{O}(n^2)$ |

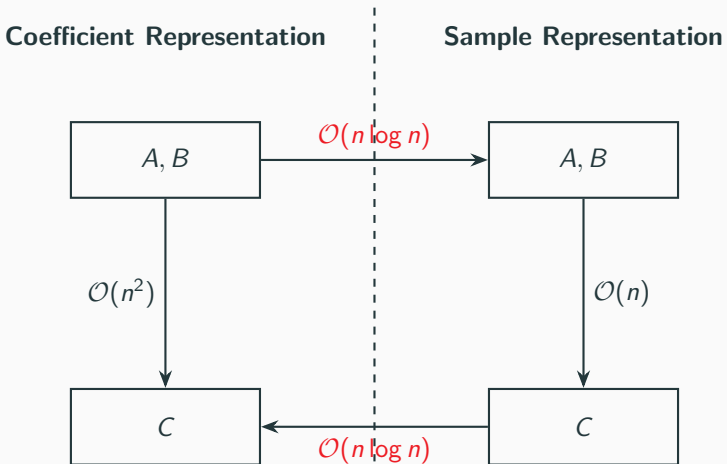
# Fast Polynomial Multiplication

---

# Fast Polynomial Multiplication

- **Problem:** Given two polynomials,  $A$  and  $B$ , in coefficient form, we need to find their product  $C$  in coefficient form.
- **Naive solution:**  $\mathcal{O}(n^2)$  time.
- **Goal:**  $\mathcal{O}(n \log n)$  time.

# High-Level Idea



- Going from coefficient representation to sample representation can be represented using a matrix multiplication:

$$\begin{pmatrix} 1 & x_0 & x_0^2 & \dots & x_0^{n-1} \\ 1 & x_1 & x_1^2 & \dots & x_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_{n-1} & x_{n-1}^2 & \dots & x_{n-1}^{n-1} \end{pmatrix} \begin{pmatrix} a_0 \\ a_1 \\ \vdots \\ a_{n-1} \end{pmatrix} = \begin{pmatrix} y_0 \\ y_1 \\ \vdots \\ y_{n-1} \end{pmatrix}$$

- Problem:** The complexity is still  $\mathcal{O}(n^2)$ .

- Given a polynomial  $A$ , and a set of points  $X$  to evaluate at, we will do the following divide and conquer algorithm.
  1. Divide  $A$  into  $A_0$  and  $A_1$  where:

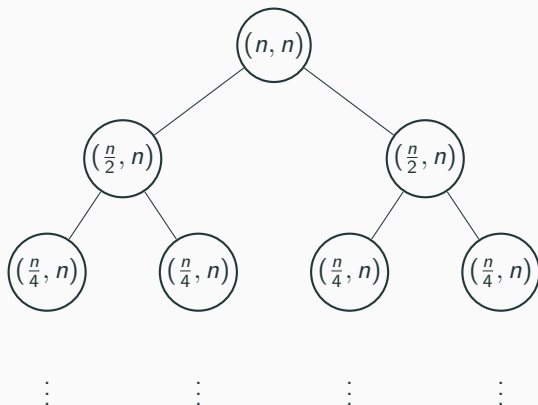
$$A_0(x) = a_0 + a_2x + a_4x^2 + \dots$$

$$A_1(x) = a_1 + a_3x + a_5x^2 + \dots$$

2. Evaluate  $A_1$  and  $A_0$  at  $x^2$  for all  $x$  in  $X$ .
3. Get  $A(x) = A_0(x^2) + x \cdot A_1(x^2)$ .



# Recursion Tree



- **Problem:** Complexity is still  $\mathcal{O}(n^2)$ .
- **Solution:** What if we can reduce the size of the set by half each time? The complexity will be  $\mathcal{O}(n \log n)$ .

# Roots of Unity

- We need to pick evaluation points that collapse when squared:

$$X = \{-1, 1\} \xrightarrow{\text{square}} \{1\}$$

$$X = \{\pm i, \pm 1\} \xrightarrow{\text{square}} \{-1, 1\}$$

$$X = \left\{ \pm \frac{1}{\sqrt{2}}(1 + i), \pm \frac{1}{\sqrt{2}}(1 - i), \pm i, \pm 1 \right\} \xrightarrow{\text{square}} \{\pm i, \pm 1\}$$

- **Roots of unity:**

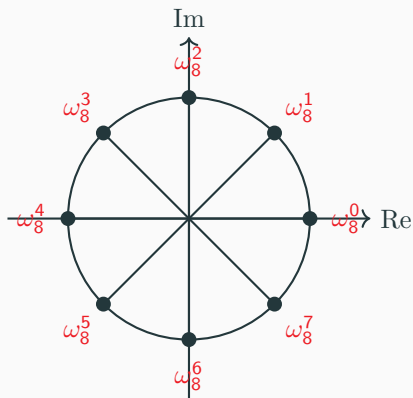
$$\omega^n = 1 = e^{i2\pi} \implies \omega = e^{i\frac{2\pi}{n}j}, \quad 0 \leq j < n$$

$$\omega_n^j = e^{i\frac{2\pi j}{n}} \text{ is called an } n\text{-th root of unity.}$$

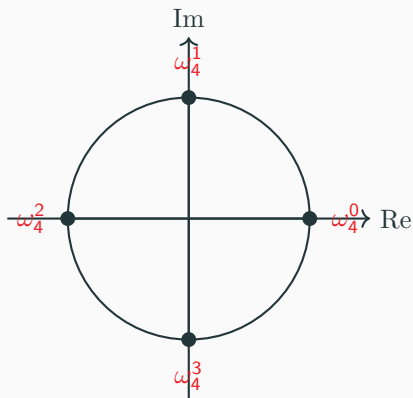
- *Note:  $n$  will always be a power of 2 in our algorithm as we are always squaring the current set of roots.*

# Roots of Unity Visualized

8th Roots of Unity ( $n = 8$ )



4th Roots of Unity ( $n = 4$ )



# The FFT Algorithm

- Assume  $\text{FFT}(A_0)$  returns  $F_0$  and  $\text{FFT}(A_1)$  returns  $F_1$ , where:

$$F_0[k] = A_0(\omega_{n/2}^k) \quad \text{and} \quad F_1[k] = A_1(\omega_{n/2}^k)$$

- Since  $\omega_{n/2}^k = \omega_n^{2k}$ , we can compute the first half:

$$F[k] = F_0[k] + \omega_n^k F_1[k]$$

- For the second half (using the property  $\omega_n^{n/2} = -1$ ):

$$\begin{aligned} F\left[k + \frac{n}{2}\right] &= A_0\left(\omega_n^{2(k+n/2)}\right) + \omega_n^{k+n/2} A_1\left(\omega_n^{2(k+n/2)}\right) \\ &= A_0(\omega_n^{2k}) + \omega_n^{n/2} \cdot \omega_n^k A_1(\omega_n^{2k}) \\ &= F_0[k] - \omega_n^k F_1[k] \end{aligned}$$

# Inverse FFT (IFFT)

- Our evaluation matrix  $V$  is now given by:

$$V_{j,k} = e^{i\frac{2\pi}{n}jk}$$

- We can easily show that its inverse matrix is:

$$V^{-1} = \frac{1}{n} V^*$$

- **IFFT Procedure:**

- Do normal FFT but evaluate at  $e^{-i\frac{2\pi}{n}jk}$
- Divide the result by  $n$  at the end.

**Questions?**

- <https://www.youtube.com/watch?v=iTMn0Kt18tg&t=3915s>
- Halim, S., Halim, F., and Effendy, S. (2020). *Competitive Programming 4, Book 2: The Lower Bound of Programming Contests in the 2020s*. Lulu Press.
- <https://cp-algorithms.com/algebra/fft.html>

Nice animated explainers:

- <https://www.youtube.com/watch?v=h7ap07q16V0>
- <https://www.youtube.com/watch?v=spUNpyF58BYpp=ygUdM2JsdWUxYnJvd24gZm91cmllciB0cmFuc2Zvcml0%3D>